

Machine Learning for ASIC Routability Prediction: A Critical Analysis of Methodological Evolution

Lang Sun
University of Toronto
Toronto, Ontario, Canada
lang.sun@mail.utoronto.ca

1 Introduction

Routability is a critical aspect of ASIC synthesis and layout. Placement optimization, whether analytical or SA-based, requires a cost function to evaluate candidate solutions, which ideally reflects the true routability. Since actual routing is computationally expensive and time consuming when it comes to large-scale designs especially in ASIC design, more accurate and efficient routability estimation methods are always in demand. Traditional estimator for this problem often fall into the "fast or accurate" dilemma. Fast estimators, such as HPWL estimation, correlate with routability but often ignore the capacity constraints and design rules, leading to inaccurate results. On the other hand, detailed estimators, such as global routing based methods, such as PathFinder [3], are accurate but often very expensive for iterative placement. Therefore, there is a need for routability estimation methods that are both global-router accurate and HPWL efficient.

Recent advances in machine learning (ML) have shown promising results in various domains, including EDA development. ML-based approaches are able to learn complex mapping from placement features to detailed routing outcomes, which shows its diversity. And its "once trained, inference fast" nature is suitable for repeated evaluation such as inner loop of simulated annealing.

To explore and analyze the ML approaches for this problem, This paper reviews three methodologies from 2014 to 2022 that show the progression of ML in this field: from expert-driven feature engineering, to architectures borrowed from computer vision, to domain-specific designs that respect circuit structure.

2 Problem Definition

In ASIC physical synthesis, routing proceeds in two stages. Global routing first tessellates the chip into a grid of rectangular gcells, where each gcell represents a coarse region with limited routing capacity, which is the number of metal tracks available for wires to pass through. The global router then constructs a routing resource graph over this grid and finds coarse paths for all nets, typically using algorithms like PathFinder that iteratively negotiate congestion by increasing costs on overused gcells. Once global routing completes, detailed routing takes over: it realizes actual metal geometries within each gcell, connecting pins with wires and vias while satisfying design rules such as minimum spacing, minimum width, via enclosure, and metal density constraints. The challenge is that congestion estimated during global routing often fails to predict detailed routing outcomes, because global routing does not model pin access blockages, via stacking effects, or the full complexity of design rules.

The congestion prediction problem aims to estimate detailed routing outcomes earlier in the flow. All three approaches reviewed

in this paper share this goal, but differ in their exact formulations. The MARS(Multivariate Adaptive Regression Splines [1])-based approach [4] takes placement plus global routing results (segment counts, via counts, pin statistics) as input, and output per-gcell utilization and DRC violation counts as regression targets. RouteNet [8] takes placement alone, encoded as a multi-channel image (RUDY (Rectangular Uniform wire DensitY)-based wire density [5], pin density, macro regions), and outputs either an overall routability classification or per-gcell hotspot predictions. LHNN [7] also works from placement, but represents the circuit as a lattice hypergraph preserving netlist connectivity, outputting both demand regression and binary congestion classification. These differences reflect the field's evolution: 1. earlier work required global routing results, while later methods predict directly from placement. 2. output formulations shifted from regression toward classification as the community recognized that identifying congestion hotspots is more actionable than predicting exact utilization values. Due to these differing formulations and benchmarks, it is impossible to compare three approaches directly and numerically, which is why this paper focuses on comparing their methodologies rather than ranking their performance.

3 MARS-based Approach

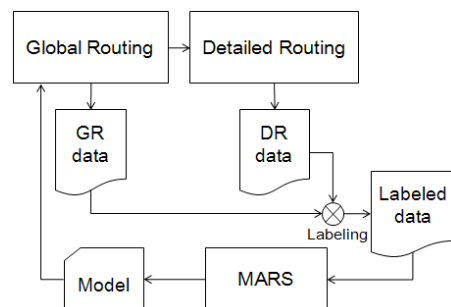


Figure 1: MARS-based congestion prediction framework.

The first approach [4], proposed by Qi et al. in 2014, follows a traditional machine learning philosophy: domain experts design features based on their understanding of what causes routing congestion, then a regression model learns the mapping from these features to congestion outcomes. The key part is not actually the model itself but the feature engineering, which is the careful selection of features that capture factors the traditional congestion models miss or cannot fully take into account.

3.1 Feature Engineering

Traditional congestion models use a simple mapping:

$$f : N \rightarrow R \quad (1)$$

where the number of wire segments N maps to resource utilization R . This ignores many factors. The authors propose a richer mapping:

$$f : (N_{seg} \times N_{via} \times \Phi_{pin}) \rightarrow (N_{DRC}, R) \quad (2)$$

where segment count, via count, and pin distribution data Φ_{pin} together predict both DRC violation count and utilization.

The authors identify two main sources of detailed routing congestion that global routing typically underestimates. The first is local pin access. Every pin needs a short wire to connect it to the routing tracks, and these access paths consume routing resources on lower metal layers. When pins are densely packed or have small shapes, accessing them becomes harder and causes more congestion. The second source is via-related blockage. Vias connect wires across metal layers, and their enclosure rules block nearby routing tracks. Stacked vias behave differently from non-stacked ones.

Based on this understanding, the authors extract features for each gcell. For pin-related information: pin count n_{pin} , pin shape size A_{pin} , and pin neighboring metric $\rho_{neighbor}$. The neighboring metric measures local crowding:

$$\rho_{neighbor} = \frac{A_{blocked}}{A_{window}} \quad (3)$$

where A_{window} is a region extending one routing pitch around the pin, and $A_{blocked}$ is the area within that window occupied by other pins or blockages. For routing-related information from global routing, they count: side-connecting segments, through-segments, stacked vias n_{stack} , and non-stacked vias $n_{nonstack}$. Since a gcell may contain multiple pins, they compute statistical aggregations—mean μ , standard deviation σ , and top- $k\%$ averages rather than just totals.

Different metal layers have different design rules, so the authors train separate models for each layer. A model for Metal2 will have different coefficients than one for Metal5.

3.2 Learning Algorithm

For the learning algorithm, the authors choose Multivariate Adaptive Regression Splines (MARS). MARS builds models of the form:

$$\hat{f}(x) = c_0 + \sum_{i=1}^N c_i \prod_{j=1}^{d_i} h_{ij}(x_{ij}) \quad (4)$$

where each $h_{ij}(x)$ is a hinge function: either $\max(0, x - t)$ or $\max(0, t - x)$ for some threshold t . The product terms with degree $d_i > 1$ capture variable interactions. For example, high pin count combined with high via count may cause more congestion than either factor alone. Unlike neural networks, MARS requires no hyperparameter tuning, and the output is a readable expression.

Once trained, the model integrates into global routing. During path search, the router queries the model to get predicted congestion for each gcell. In PathFinder terms, the node cost C_n becomes:

$$C_n = \begin{cases} \text{utilization} & \text{if } n_{DRC} = 0 \\ 1 + \frac{n_{DRC}}{N_m} & \text{otherwise} \end{cases} \quad (5)$$

where N_m is a normalization constant. This replaces the traditional overflow-based cost with a learned prediction of detailed routing outcome.

3.3 Results and Limitations

The approach was tested on DAC 2012 benchmarks at 45nm. When integrated into a global router, it reduced DRC violations by 34-37% and via count by up to 13%, depending on the routing flow configuration. Feature importance analysis showed pin count and segment counts were most influential, confirming the authors' intuition.

However, this approach has limitations. It requires global routing results as input, which means it cannot predict from placement alone. Also, the features are designed for a specific technology and may need re-engineering for different processes. Moreover, while MARS captures non-linearity, it may miss patterns that experts did not anticipate.

Algorithm 1 MARS-based Congestion Prediction

Require: Placement P , Global routing result GR

Ensure: Per-gcell DRC prediction, utilization prediction

```

1: Feature Extraction:
2: for each gcell  $g$  do
3:   Extract pin features:  $n_{pin}, A_{pin}, \rho_{neighbor}$ 
4:   Extract routing features from  $GR$ : segment counts, via counts
5:   Compute statistics:  $\mu, \sigma$ , top- $k\%$  averages
6: end for
7: Model Training (per metal layer):
8: Collect labeled data:  $(features, DRC_{actual}, util_{actual})$ 
9: Train MARS model:  $\hat{f}(x) = c_0 + \sum_i c_i \prod_j h_{ij}(x)$ 
10: Integration into Global Routing:
11: for each PathFinder iteration do
12:   for each gcell  $g$  in path search do
13:      $n_{DRC}, util \leftarrow \hat{f}(features_g)$ 
14:      $C_g \leftarrow util$  if  $n_{DRC} = 0$ , else  $1 + n_{DRC}/N_m$ 
15:   end for
16:   Update paths using predicted costs
17: end for
```

4 RouteNet: Image-based Approach

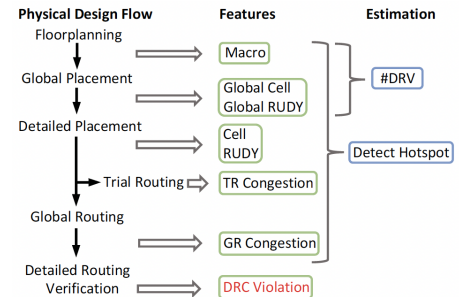


Figure 2: RouteNet image-based architecture.

The second approach [8], RouteNet proposed by Xie et al. in 2018, takes a fundamentally different direction: instead of hand-crafting features, treat the circuit layout as an image and let a convolutional neural network (CNN) learn the patterns automatically. This borrows proven architectures from computer vision, where CNNs have dominated image recognition tasks since 2012.

4.1 Image Representation

The core insight of this approach is that placement data naturally forms a 2D grid structure—similar to pixels in an image. The authors represent the layout as a multi-channel image. For overall DRV (Design Rule Violations) prediction, three channels are used:

- **Macro regions:** binary mask indicating macro blockages
- **Long-range RUDY:** wire density from nets spanning longer than a threshold
- **RUDY pins:** pin density weighted by connected nets' RUDY

RUDY extends HPWL by spreading each net's wire demand uniformly within its bounding box. For the k -th net with bounding box corners (x_{min}^k, y_{min}^k) to (x_{max}^k, y_{max}^k) :

$$w^k = x_{max}^k - x_{min}^k, \quad h^k = y_{max}^k - y_{min}^k \quad (6)$$

$$RUDY^k(x, y) \propto \begin{cases} \frac{w^k + h^k}{w^k \times h^k} & \text{if } (x, y) \in \text{bounding box} \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

The total RUDY at any location sums over all nets:

$$RUDY(x, y) = \sum_{k=1}^K RUDY^k(x, y) \quad (8)$$

Long-range RUDY includes only nets above a distance threshold, as these correlate more strongly with DRV.

4.2 Network Architecture

RouteNet addresses two prediction tasks with different architectures and different inputs. For overall DRV prediction, which works from placement alone without routing, the authors use ResNet-18 pretrained on ImageNet. The input tensor $X_i \in \mathbb{R}^{w \times h \times 3}$ is resized to $224 \times 224 \times 3$ to match ImageNet dimensions. Placements are grouped into four classes c_0, c_1, c_2, c_3 based on DRV quartiles, and training uses only extreme classes (c_0 and c_3) for cleaner decision boundaries.

For DRC hotspot detection, RouteNet uses global routing congestion as an additional input feature. The output must be a dense map indicating hotspot locations—not a single score. This requires a fully convolutional network (FCN):

$$f_{hotspot} : \mathbb{R}^{w \times h \times F} \rightarrow \mathbb{R}^{w \times h} \quad (9)$$

The FCN uses two pooling layers to downsample feature maps to $\frac{h}{4} \times \frac{w}{4}$, then transposed convolutions to upsample back to full resolution. A shortcut connects the 2nd layer directly to the 7th layer, preserving both local detail and global context. The loss function combines pixel-wise error with L2 regularization:

$$Loss = \sum_{i=1}^N \sum_{m,n} \|f_{hotspot}(X_{imn}) - Y_{imn}^{clip}\|^2 + \lambda \|W\|^2 \quad (10)$$

where Y^{clip} clips extreme DRV values to reduce outlier effects.

The core design of this approach is about the receptive field. A congestion hotspot in one gcell may be caused by distant macros or routing demand—the detailed router may detour around congestion elsewhere. Traditional methods like SVM or logistic regression extract features from local windows; experiments found 5×5 gcells optimal, but accuracy remained limited. CNNs expand receptive field through pooling: deeper layers aggregate information from larger regions. Further ablation studies confirm this: removing pooling layers significantly degrades accuracy.

Algorithm 2 RouteNet #DRV Prediction

Require: Placements $\{X_i \in \mathbb{R}^{w \times h \times 3}\}_{i=1}^N$, Targets $\{y_i\}_{i=1}^N$

Ensure: Trained model $f_{\#DRV}$

- 1: **Preprocess:**
 - 2: **for** each $i \in [1, N]$ **do**
 - 3: Resize X_i to $X_i^{\#DRV} \in \mathbb{R}^{224 \times 224 \times 3}$
 - 4: **end for**
 - 5: Find quartiles q_1, q_2, q_3 from $\{y_i\}$
 - 6: Assign class $C_i \in \{c_0, c_1, c_2, c_3\}$ by quartile
 - 7: Training set $\leftarrow \{(X_i, C_i) \mid C_i = c_0 \text{ or } C_i = c_3\}$
 - 8: **Training:**
 - 9: Load pretrained ResNet-18
 - 10: Replace output layer for single score
 - 11: Train with MSE loss, SGD optimizer, ~ 30 epochs
-

4.3 Results and Limitations

RouteNet was tested on ISPD 2015 benchmarks with mixed-size designs. For overall DRV prediction (without routing), it achieves accuracy similar to global routing while being orders of magnitude faster—under one second per placement. For hotspot detection (with GR features), it improves true positive rate (TPR) by 50% over global routing at matched false positive rate (FPR). Both tasks outperform SVM and logistic regression baselines.

However, the image representation has a fundamental limitation: it discards netlist topology. RUDY and density maps encode spatial distribution, but not which cells share the same net. A net spanning distant gcells creates correlated congestion at both ends, but an image-based model sees only spatial proximity, not topological connectivity, which is addressed directly by the next approach.

5 LHNN: Graph-based Approach

The third approach [7], LHNN proposed by Wang et al. in 2022, addresses the fundamental limitation of image-based methods, which is neglecting netlist topology. The key insight is circuits are not images, they are hypergraphs. A net connecting multiple pins creates dependencies across all the gcells it spans. When we convert a circuit to RUDY maps or density images, this connectivity information is lost. Two gcells may both show high wire density, but the image does not tell us whether they share the same net and will experience correlated congestion.

Consider a concrete example: a net spans gcells at opposite corners of the chip. In an image representation, these gcells are distant pixels. For a CNN to relate them, information must propagate through many convolutional layers—and even then, the network

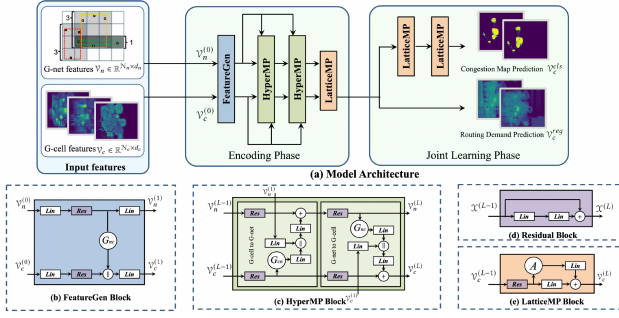


Figure 3: LHNN lattice hypergraph neural network architecture.

does not know they belong to the same net. In contrast, LHNN represents this net as a hyperedge connecting both gcells directly. Information flows between them in a single message-passing step, regardless of physical distance.

5.1 LH-Graph Formulation

LHNN introduces the Lattice Hypergraph (LH-graph), which combines two graph structures. The lattice graph captures spatial adjacency: gcells that are physical neighbors can share information. The hypergraph captures topological connectivity: gcells covered by the same net can share information. Together, they preserve both the 2D layout structure and the netlist structure.

Formally, the LH-graph is defined as $G = (V_c, V_n, A, H)$ where:

- $V_c \in \mathbb{R}^{N_c \times d_c}$: features for N_c gcells
- $V_n \in \mathbb{R}^{N_n \times d_n}$: features for N_n nets
- $A \in \mathbb{R}^{N_c \times N_c}$: lattice adjacency matrix, where $A_{ij} = 1$ if gcells i and j are spatial neighbors
- $H \in \mathbb{R}^{N_c \times N_n}$: hypergraph incidence matrix, where $H_{ij} = 1$ if gcell i is covered by net j

This formulation parallels the hypergraph representation used in partitioning algorithms like hMetis [2], where nets are hyperedges connecting multiple cells. The difference is that LHNN uses this structure for learning, not optimization.

Each net node carries four features: vertical span $span_V$, horizontal span $span_H$, pin count $npin$, and bounding box area $Area = span_H \times span_V$. Each gcell node carries net density, pin density, and a terminal cell mask. Importantly, conventional features like RUDY can be recovered from this graph by simple aggregation:

$$RUDY_i \approx \sum_{j: H_{ij}=1} \frac{npin_j \times (span_H^j + span_V^j)}{Area_j} \quad (11)$$

This shows that the graph formulation subsumes hand-crafted features while enabling the network to learn richer representations.

5.2 Network Architecture

The LHNN architecture has two phases: encoding and joint learning. The encoding phase uses three types of message-passing blocks, each serving a distinct purpose.

Algorithm 3 LHNN Congestion Prediction

Require: Placement P , Netlist N

Ensure: Per-gcell congestion classification, demand regression

- 1: **Graph Construction:**
- 2: Build lattice adjacency A : $A_{ij} = 1$ if gcells i, j adjacent
- 3: Build hypergraph incidence H : $H_{ij} = 1$ if gcell i in net j
- 4: Initialize $V_c^{(0)}$ (gcell features), $V_n^{(0)}$ (net features)
- 5: **Encoding Phase:**
- 6: $V_c^{(1)} \leftarrow \phi_c(f_c(V_c^{(0)}) \| H \cdot f_n(V_n^{(0)}))$ ▷ FeatureGen
- 7: **for** $l = 1$ to L_{hyper} **do** ▷ HyperMP blocks
- 8: $V_n^{(l)} \leftarrow$ aggregate from gcells via $H^\top$ (topological)
- 9: $V_c^{(l)} \leftarrow$ aggregate from nets via H (topological)
- 10: **end for**
- 11: **for** $l = 1$ to $L_{lattice}$ **do** ▷ LatticeMP blocks
- 12: $V_c^{(l)} \leftarrow$ aggregate from neighbors via A (geometric)
- 13: **end for**
- 14: **Joint Learning Phase:**
- 15: $\hat{y}^{reg} \leftarrow MLP_{reg}(V_c^{(L)})$ ▷ demand regression
- 16: $\hat{y}^{cls} \leftarrow MLP_{cls}(V_c^{(L)})$ ▷ congestion classification
- 17: $\mathcal{L} \leftarrow MSE(\hat{y}^{reg}, y^{reg}) + BCE_Y(\hat{y}^{cls}, y^{cls})$

The **FeatureGen block** initializes the learning process. It transforms raw features through MLPs and performs an initial aggregation from nets to gcells:

$$V_c^{(1)} = \phi_c^{(1)} \left(f_c^{(1)}(V_c^{(0)}) \| H \cdot f_n^{(1)}(V_n^{(0)}) \right) \quad (12)$$

where $\|$ denotes concatenation. This gives each gcell an initial representation that includes information from all nets covering it.

The **HyperMP blocks** expand the receptive field topologically. They alternate message passing in two directions: gcells aggregate information to their nets (via $H^\top$), then nets distribute information back to their gcells (via H). After one HyperMP block, each gcell knows about other gcells that share any of its nets. After two blocks, it knows about gcells two hops away in the hypergraph. This is the key mechanism that allows distant but topologically connected gcells to influence each other's predictions.

The **LatticeMP blocks** expand the receptive field geometrically. They pass messages between spatially adjacent gcells using the lattice adjacency matrix A . This captures local spatial patterns that the hypergraph might miss. For example, congestion spreading to neighboring gcells even if they do not share nets.

This architecture uses 2 HyperMP blocks followed by 3 LatticeMP blocks, with hidden dimension 32 throughout.

5.3 Joint Learning

The second phase is joint learning phase, which addresses a practical problem. One could train a model to predict routing demand (regression), then threshold the predictions to identify congested gcells. But this works poorly because demand values in congested and non-congested regions are often very close, meaning small prediction errors lead to many misclassifications.

Table 1: Comparison of ML-based Congestion Prediction Approaches

Aspect	MARS-based	RouteNet	LHNN
Year	2014	2018	2022
Representation	Feature vector	Image	Hypergraph
Input stage	After GR	After placement	After placement
Netlist topology	Indirect (via features)	Lost	Preserved
Spatial structure	Lost	Preserved	Preserved
Interpretability	High (readable expr.)	Low	Low
Technology transfer	Requires re-engineering	Transfer learning	Graph structure universal

Instead, LHNN jointly predicts both routing demand and congestion:

$$\mathcal{L} = \mathcal{L}_{reg} + \mathcal{L}_{cls} \quad (13)$$

The regression loss is standard MSE. The classification loss uses weighted binary cross-entropy with a factor $\gamma \in (0, 1]$ that reduces the contribution of non-congested gcells, addressing the class imbalance (roughly 83% of gcells are non-congested). The regression task provides a richer training signal; the classification task produces the actionable output.

5.4 Results and Insights

LHNN was evaluated on ISPD 2011 and DAC 2012 benchmarks, with placements from DREAMPlace and ground truth from NCTU-GR 2.0. It achieves over 35% improvement in F1 score compared to image-based models like U-net and Pix2Pix.

The ablation studies reveal what matters most: removing HyperMP edges causes the largest performance drop. This confirms that topological message passing, not just deeper networks, is essential. It is also interesting that a simple 4-layer MLP on local features performs comparably to U-net, suggesting that CNNs’ geometric receptive field expansion adds little value without topological structure.

The methodology actually aligns with a principle from lecture 9, which is algorithms that exploit problem structure outperform generic approaches. Just as hMetis exploits hypergraph connectivity for partitioning, LHNN exploits it for congestion prediction.

6 Critical Analysis

The three approaches represent a progression in how to model circuits for machine learning. The core question is not which model to use (MARS, CNN, or GNN), but rather what information about the circuit to preserve during learning. Table 1 summarizes the key differences; here we focus on the trade-offs each methodology makes.

It is well demonstrated from the course material that good algorithms exploit problem structure. PathFinder succeeds by modeling routing as negotiated resource sharing. hMetis succeeds by exploiting hypergraph connectivity for partitioning. The evolution from MARS to LHNN reflects this principle: image-based methods impose a 2D grid structure that discards the circuit’s true hypergraph nature. LHNN’s improvement suggests that preserving netlist topology is essential. Moreover, ML-based congestion prediction can be viewed as learning a better cost function, one that approximates detailed routing outcomes at placement-time speed, filling the gap

between fast-but-inaccurate HPWL and accurate-but-slow global routing.

6.1 Trade-offs

Interpretability vs. Accuracy. MARS produces a readable expression; one can inspect coefficients and identify which features matter (e.g., pin count has importance 30, segment count 29). Neural networks are black boxes, which means we know they work, but cannot easily explain why a particular gcell is predicted congested. For integration into existing CAD flows, interpretability aids debugging and trust.

Prediction Timing. The MARS-based approach requires global routing results, limiting its use to refining GR decisions. RouteNet and LHNN predict from placement alone, enabling earlier intervention. This “predict early” capability is valuable: catching congestion before routing saves significant runtime in iterative optimization.

Generalization. MARS features are technology-specific as features designed for 45nm may not transfer to 7nm. RouteNet uses ImageNet pretraining but the domain gap between natural images and circuits is large. LHNN’s graph structure is arguably more universal. Netlists are hypergraphs regardless of technology node. However, this remains to be validated across diverse industrial designs and technologies as this is a realitively new reaseach.

6.2 Limitations and Future Directions

All three approaches share limitations: they require expensive ground truth from detailed routing for training. Currently, they are validated primarily on academic benchmarks rather than industrial designs with billions of cells, and predict congestion without directly optimizing it.

Future directions from these works include richer graph structures incorporating timing and power, multi-task learning for specific DRC types, and tighter integration with placement optimization algorithms.

Additionally, global awareness remains a challenge. LHNN addresses this through hypergraph connectivity, but further improvements may come from **attention mechanisms**, such as Graph Attention Networks [6] (GAT) or Transformer-style self attention, which could further improve global awareness. The evolution from CNNs to hypergraph message passing shows that capturing long-range dependencies is critical; attention mechanisms offer another path to achieve this by learning which nets or gcells are most relevant for predicting congestion at each location, rather than treating all neighbors equally during aggregation.

7 Conclusion

This paper surveyed three machine learning approaches for ASIC routing congestion prediction, tracing a methodological evolution from expert-crafted features (MARS, 2014), to image-based CNNs (RouteNet, 2018), to domain-specific hypergraph networks (LHNN, 2022).

The central insight from this progression is that representation matters more than model complexity. LHNN’s 35% improvement over CNNs comes not from a more powerful architecture, but from preserving the hypergraph structure that defines circuit connectivity. This aligns with a recurring theme in CAD: algorithms succeed when they respect problem structure. This ideology has pushed the approaches borrowed from other field (e.g., computer vision) to more domain-specific designs.

As for future directions, the field is moving toward earlier prediction, richer representations, and mechanisms for global awareness, whether through hypergraph topology or attention-based learning, the ultimate goal remains open: not just predicting routability, but using learned models to fully guide or even automate the placement and routing toward manufacturable designs and we will all be out of jobs then.

References

- [1] Jerome H. Friedman. 1991. Multivariate Adaptive Regression Splines. *The Annals of Statistics* 19, 1 (1991), 1–67.
- [2] George Karypis, Rajat Aggarwal, Vipin Kumar, and Shashi Shekhar. 1997. Multi-level Hypergraph Partitioning: Applications in VLSI Domain. In *Proceedings of the 34th Design Automation Conference (DAC)*. ACM/IEEE, 526–529. doi:10.1109/DAC.1997.597203
- [3] Larry McMurchie and Carl Ebeling. 1995. PathFinder: A Negotiation-Based Performance-Driven Router for FPGAs. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. ACM, Monterey, CA, USA, 111–117. doi:10.1145/201310.201328
- [4] Zhongdong Qi, Yici Cai, and Qiang Zhou. 2014. Accurate Prediction of Detailed Routing Congestion Using Supervised Data Learning. In *Proceedings of the IEEE International Conference on Computer Design (ICCD)*. IEEE, 97–103. doi:10.1109/ICCD.2014.6974667
- [5] Peter Spindler and Frank M. Johannes. 2007. Fast and Accurate Routing Demand Estimation for Efficient Routability-Driven Placement. In *Proceedings of the Design, Automation and Test in Europe Conference (DATE)*. IEEE, 1–6. doi:10.1109/DATE.2007.364463
- [6] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*.
- [7] Bowen Wang, Guibao Shen, Dong Li, Jianye Hao, Wulong Liu, Yu Huang, Hongzhong Wu, Yibo Lin, Guangyong Chen, and Pheng Ann Heng. 2022. LHNN: Lattice Hypergraph Neural Network for VLSI Congestion Prediction. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*. ACM, San Francisco, CA, USA, 1297–1302. doi:10.1145/3489517.3530675
- [8] Zhiyao Xie, Yu-Hung Huang, Guan-Qi Fang, Haoxing Ren, Shao-Yun Fang, Yiran Chen, and Jiang Hu. 2018. RouteNet: Routability Prediction for Mixed-Size Designs Using Convolutional Neural Network. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD)*. ACM, San Diego, CA, USA, 1–8. doi:10.1145/3240765.3240843